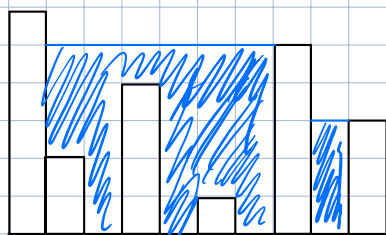
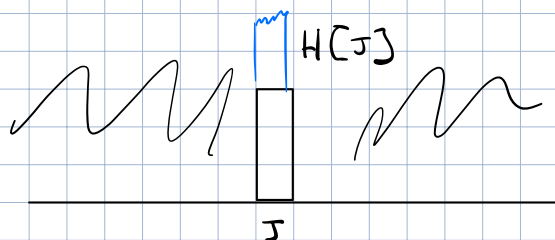


Trapping Rain Water

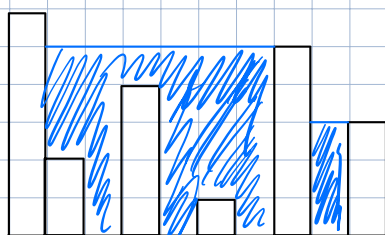


H 6, 2, 0, 4, 0, 1, 0, 5, 0, 3

$$\max \{ H[1 \dots j-1] \} \quad \dots \quad \max \{ H[j+1 \dots n] \}$$



$$wr[j] = \min \left\{ \begin{array}{l} \max \{ H[1 \dots j-1] \}, \\ \max \{ H[j+1 \dots n] \} \end{array} \right\} - H[j]$$



H 6, 2, 0, 4, 0, 1, 0, 5, 0, 3

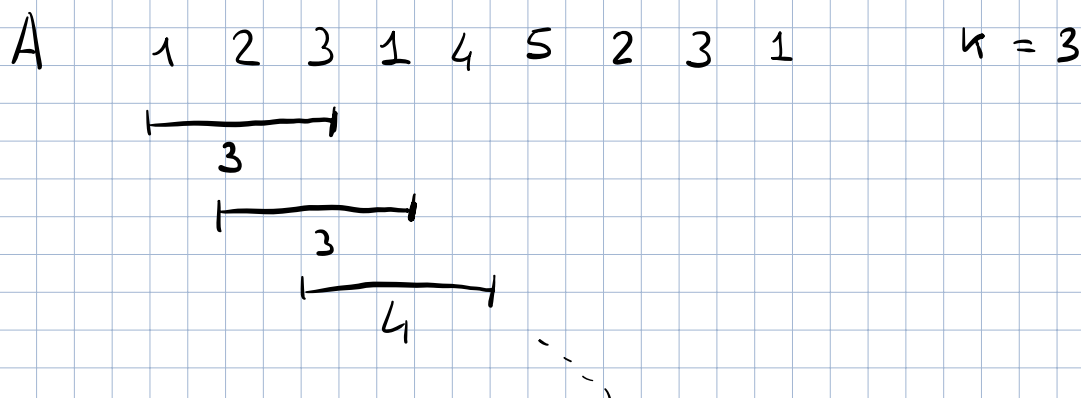
max S 6 5 5 5 5 5 5 5 3 3

max P 6 6 6 5 6 6 6 6 6 6
↑

$$wr[5] = 5 - 0 = 5$$

SLIDING WINDOW MAXIMA

Given $A[1, n]$ and an integer k ,
find the largest element of each window of
size k



Trivial Solution

$$\Theta(n \cdot k)$$

$r = []$

for $i \in [1, n - k]$

$M = A[i]$

for $j \in [i, i + k]$

$M = \max(M, A[j])$

$r.append(M)$

Better Solution

A 1 2 3 ~~4~~ 4 5 (2) 3 1

The diagram shows a sequence of numbers: 1, 2, 3, 4, 4, 5, 2, 3, 1. The first 4 is crossed out with a red 'X'. The 2 is circled in red. Below the sequence, there are two horizontal brackets. The first bracket starts under the first 4 and ends under the second 4. The second bracket starts under the second 4 and ends under the 2. This indicates that the elements from index 4 to the end of the array are shifted one position to the right to accommodate the new element 2 at index 6.

- A (multi)set of k elements
- At each iteration we need
 - Insert a new element
 - Delete an old element
 - Compute the max

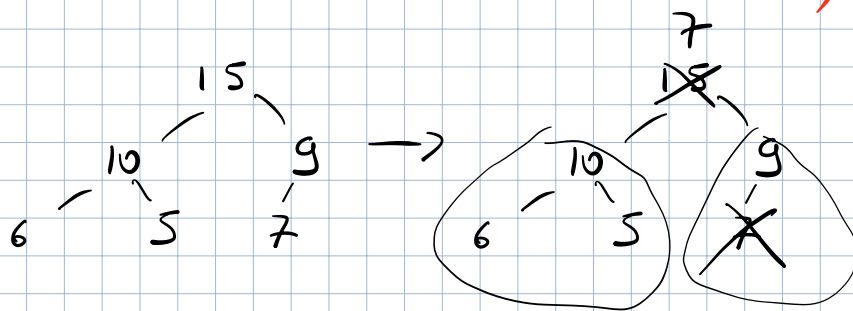
Big Question: what's the best data structure supporting these ops?

Max Heap

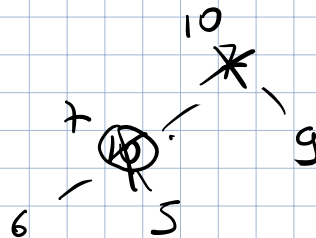
Store a (multi) set S , supporting

- Extract Max $\Theta(\log |S|)$
- Max $O(1)$
- Insert (x) $\Theta(\log |S|)$

we cannot delete arbitrary elements*
(*easily)



Heapify



Lazy Delete

We store pairs $\langle \text{value}, \text{position} \rangle$ in the heap

At every iteration

- we insert the new element
- while curr_max is outside the window
 - extract_max
- report curr_max

Time complexity is dominated by

$\text{cost_of_insert} + \text{cost_of_extract_max}$

n insertions $\leq n$ extract_max

so, $\Theta(n \log n)$ time

every element is inserted exactly once and deleted at most once

Predecessor Problem

(B)BST

Store a set S , supporting

- Insert (x)

- Delete (x)

- Search (x)

Dictionary
Problem

Hash Map

- Max/Min

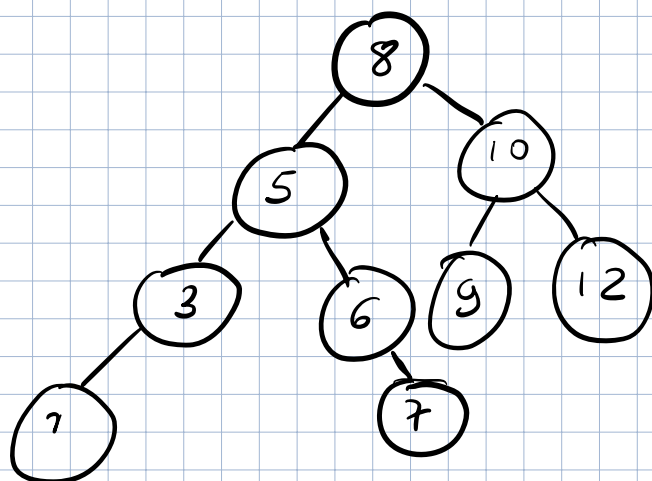
- Successor (x) $\min \{ y \mid y \in S \text{ AND } y \geq x \}$

- Predecessor (x)

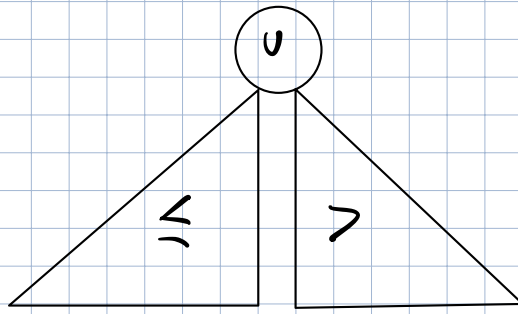
All ops in $\Theta(\log |S|)$

SWM in $\Theta(n \log k)$ time

1 3 5 6 - 8 9 10 12

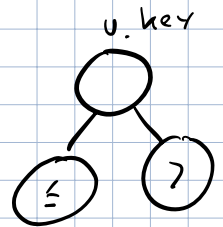


BST property



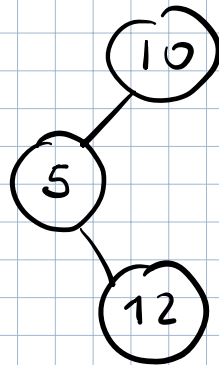
Is a given binary tree T a BST?

for every node u , check if



$u.\text{left.key} \leq u.\text{key} < u.\text{right.key}$

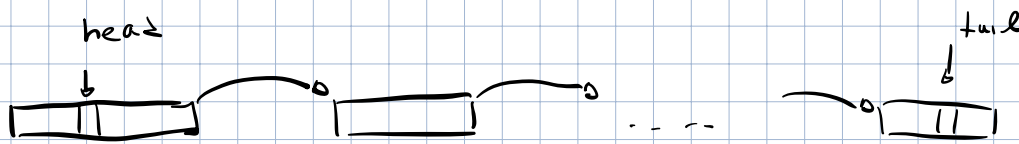
NO



$\leftarrow$ is_BST?, min, max
 $p(u)$
 if $u == \text{NULL}$
 return True, $+\infty$, $-\infty$
 $rl, ml, Ml = p(u.\text{left})$
 $rr, mr, Mr = p(u.\text{right})$
 $ru = Ml \leq u.\text{key} < mr$ AND
 rl AND rr
 $mu = \min(u.\text{key}, ml, mr)$
 $Mu = \max(u.\text{key}, Ml, Mr)$
 return ru, mu, Mu

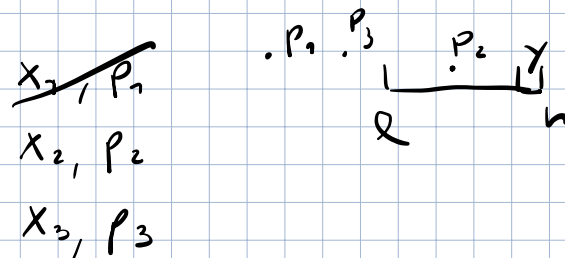
Optimal Solution $\Theta(n)$ time

It uses a Deque



Algorithm

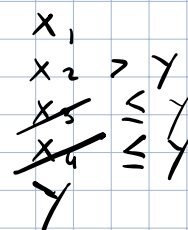
- keep a deque Q storing pairs $\langle e, p \rangle$
- Repeat n times
 - move the window one position to right
 - remove from head of Q **any element** outside the current window



we stop as soon as the current head of Q is inside the window

- Insert the current element y from tail of Q

Moving y up till the current tail is $\leq y$



- report the head of Q as maximum of the window

Time complexity

of removes + # of pushes

$\leq n$

$\parallel$
 n

$\Theta(n)$

Correctness

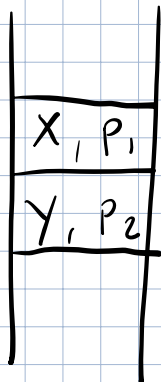
Property #1

Q is sorted in decreasing order

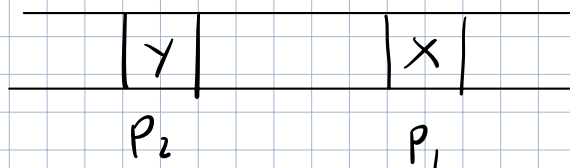
Proof

Assume Q is not sorted
 $x < y$

Q

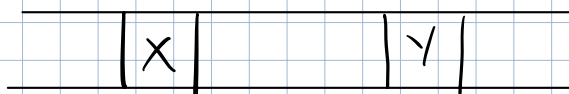


• if $p_1 > p_2$



not possible

• if $p_1 < p_2$



y would kill x



An element is a **RightLeader**

if and only if

the element is larger than any element to its right

4 2 3 1 1

Property #2

At every iteration Q contains all and only the RightLeaders in the window

Proof

① **ALL** A RightLeader cannot be removed

② **ONLY**

It's not a RightLeader

↙
| b | | a |

$b \leq a$

b cannot be in Q